Guia Completo do Widicom - Metabuscador de Lost Media

Visão Geral

O **Widicom** é um sistema completo de busca especializado em Lost Media e arquivos raros hospedados em serviços de nuvem públicos. O projeto é composto por dois componentes principais: uma **API RESTful em Node.js** que realiza a busca e um **Frontend em React** que fornece a interface do usuário.

Este guia detalha como configurar, executar e utilizar ambos os componentes de forma integrada.

Arquitetura do Sistema

O Widicom opera em uma arquitetura cliente-servidor onde o frontend comunica-se com a API através de requisições HTTP. O fluxo de dados segue este padrão:

1. **Entrada do Usuário**: O usuário insere um termo de busca na interface do frontend.
2. **Requisição HTTP**: O frontend envia uma requisição GET para a API com o termo de busca como parâmetro.
3. **Processamento na API**: A API gera Google Dorks, executa buscas assíncronas em múltiplas plataformas, resolve links diretos e verifica a validade dos links.
4. **Resposta JSON**: A API retorna um JSON estruturado com os resultados encontrados.
5. **Renderização**: O frontend processa a resposta e exibe os resultados de forma visual e organizada.

Instalação e Configuração

Pré-requisitos

Certifique-se de ter instalado em seu sistema:

- **Node.js** (versão 16.x ou superior)
- **npm** ou **pnpm** (gerenciador de pacotes)
- **Git** (para clonar repositórios, se necessário)

Passo 1: Configurar a API

A API é o coração do Widicom e deve ser iniciada primeiro.

1.1 Navegar para o diretório da API

```bash
cd /caminho/para/lost-media-metasearcher
```

1.2 Instalar dependências

```bash
npm install
```

As dependências principais incluem:

Pacote	Versão	Propósito
`fastify`	^4.24.3	Framework web de alta performance
`axios`	^1.6.2	Cliente HTTP para requisições
`cheerio`	^1.0.0-rc.12	Parsing de HTML
`dotenv`	^16.3.1	Gerenciamento de variáveis de ambiente
`fastify-cors`	^6.1.0	Suporte a CORS

1.3 Configurar variáveis de ambiente

Crie um arquivo `.env` na raiz do projeto da API:

```plain text
PORT=3000
HOST=0.0.0.0
NODE_ENV=development
```

1.4 Iniciar a API

Modo de desenvolvimento (com auto-reload):

```bash
npm run dev
```

Modo de produção:

```bash
npm start
```

A API estará disponível em `http://localhost:3000`. Você verá uma mensagem confirmando que o servidor está rodando.

Passo 2: Configurar o Frontend (Widicom)

O frontend é a interface visual que consome a API.

2.1 Navegar para o diretório do frontend

```bash
cd /home/ubuntu/widicom
```

2.2 Instalar dependências

```bash
pnpm install
```

2.3 Configurar a URL da API

O frontend precisa saber onde encontrar a API. Existem duas formas de configurar:

Opção A: Variável de Ambiente

Crie um arquivo `.env.local` na raiz do projeto frontend:

```plain text
VITE_API_URL=http://localhost:3000
```

Opção B: Modificar o arquivo de configuração

Edite o arquivo `client/src/lib/api.ts` e altere a linha:

```typescript
const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:3000';
```

Para apontar para a URL correta da API.

2.4 Iniciar o Frontend

Modo de desenvolvimento:

```bash
pnpm dev
```

O frontend estará disponível em `http://localhost:5173` (ou outra porta se 5173 estiver ocupada).

Uso da Interface

Busca Básica

1. Acesse a interface do Widicom em `http://localhost:5173` .
2. Digite um termo de busca na barra de pesquisa (ex: "The Day the Earth Stood Still 1951").
3. Pressione **Enter** ou clique no botão **Buscar**.
4. Aguarde enquanto a API busca em múltiplas plataformas simultaneamente.

Interpretando os Resultados

Cada resultado é exibido em um card contendo:

- **Título**: Nome do arquivo ou conteúdo encontrado.
- **Plataforma**: Serviço de nuvem onde o arquivo está hospedado (Google Drive, Mega.nz, etc.).
- **Status**: Indica se o link está Ativo, Inativo ou Desconhecido.
- **URL Original**: Link direto para a página de visualização na plataforma.
- **URL de Download**: Link otimizado para download direto (quando disponível).
- **Botões de Ação**: Opções para abrir a página original ou fazer download.

Dicas de Busca Eficaz

O Widicom utiliza um motor de dorking avançado que gera múltiplas variações de busca automaticamente. Para obter melhores resultados:

- Use termos específicos e descritivos (ex: "The Phantom Menace 1999 DVD" em vez de apenas "Star Wars").
- Tente variações do título (ex: "Filme ABC", "ABC Movie", "ABC Film").
- Inclua o ano de lançamento quando relevante.
- Busque por formatos específicos (ex: "filme.mkv", "arquivo.zip").

Estrutura da API

Endpoints Disponíveis

GET `/`

Verifica o status da API.

Resposta de Sucesso (200):

JSON

```
{
  "message": "API de Metabuscador de Lost Media está ativa!",
  "status": "online",
  "version": "1.0.0"
}
```

GET `/search?query=termo`

Realiza uma busca de Lost Media.

Parâmetros:

Parâmetro	Tipo	Obrigatório	Descrição
`query`	string	Sim	Termo de busca

Resposta de Sucesso (200):

JSON

```
{
  "query": "The Day the Earth Stood Still",
  "total_resultados": 5,
  "resultados": [
    {
      "titulo": "The Day the Earth Stood Still (1951) - Full Movie",
      "plataforma": "Google Drive",
      "url_original": "https://drive.google.com/file/d/ABC123/view",
      "url_download_direto": "https://drive.google.com/uc?export=download&id=AB
      "status": "Ativo"
    },
    {
      "titulo": "The Day the Earth Stood Still 1951 DVDRip",
      "plataforma": "Mega.nz",
      "url_original": "https://mega.nz/file/ABC123",
      "url_download_direto": "https://mega.nz/file/ABC123",
      "status": "Desconhecido"
    }
  ]
}
```

Resposta de Erro (400):

JSON

```
{
  "error": "O parâmetro \"query\" é obrigatório."
}
```

Fluxo de Processamento da API

A API segue um pipeline bem definido para processar cada busca:

1. **Geração de Dorks** (`dorkEngine.js`): Transforma o termo de busca em múltiplas Google Dorks otimizadas para diferentes plataformas.
2. **Busca Assíncrona** (`scraperService.js`): Executa requisições paralelas para instâncias públicas do SearxNG, extraindo URLs potenciais.
3. **Resolução de Links** (`linkResolver.js`): Converte URLs de visualização em links de download direto (quando possível). Exemplo: Google Drive view → Google Drive export/download.
4. **Verificação de Links** (`linkChecker.js`): Realiza requisições HEAD para validar se cada link está ativo (HTTP 200) ou morto (HTTP 404).

5. **Formatação e Resposta**: Retorna um JSON estruturado com todos os resultados validados.

Configuração Avançada

Usar Instâncias Diferentes do SearxNG

O arquivo `src/services/scraperService.js` contém um array de instâncias públicas do SearxNG:

```JavaScript
const SEARXNG_INSTANCES = [
  'https://searx.be',
  'https://searx.me',
  'https://searx.space'
];
```

Para adicionar mais instâncias ou usar instâncias privadas:

1. Edite o arquivo `src/services/scraperService.js`.
2. Adicione as URLs das instâncias ao array `SEARXNG_INSTANCES`.
3. Reinicie a API.

Integração com SerpAPI (Opcional)

Para usar a SerpAPI em vez do SearxNG (oferece mais resultados, mas requer chave de API):

1. Obtenha uma chave de API em https://serpapi.com.
2. Adicione a chave ao arquivo `.env` da API:

```Plain Text
SERPAPI_KEY=sua_chave_aqui
```

1. Modifique o `src/services/scraperService.js` para usar a SerpAPI.

Renderização JavaScript com Puppeteer (Opcional)

Algumas plataformas como Mega.nz e MediaFire requerem renderização de JavaScript para obter links diretos. Para ativar esse recurso:

1. Instale o Puppeteer:

```Bash
```

```
npm install puppeteer
```

1. Modifique o `src/services/linkResolver.js` para usar Puppeteer quando necessário.

Troubleshooting

Problema: "API não disponível" no Frontend

Causa: A API não está rodando ou a URL está incorreta.

Solução:

1. Verifique se a API está rodando: `npm run dev` no diretório da API.
2. Confirme que a porta 3000 está disponível.
3. Verifique a configuração de `VITE_API_URL` no frontend.

Problema: "Nenhum resultado encontrado"

Causa: A busca não retornou resultados ou as instâncias do SearxNG estão indisponíveis.

Solução:

1. Tente uma busca com termos mais genéricos.
2. Verifique a disponibilidade das instâncias do SearxNG visitando-as no navegador.
3. Consulte os logs da API para mais detalhes: `npm run dev` mostra logs em tempo real.

Problema: Links retornam status "Desconhecido"

Causa: O link checker não conseguiu verificar o status do link (timeout, bloqueio, etc.).

Solução:

1. Tente acessar o link manualmente no navegador.
2. Alguns servidores bloqueiam requisições HEAD; o link pode estar ativo mesmo com status "Desconhecido".
3. Aumente o timeout no `src/services/linkChecker.js` se necessário.

Estrutura de Arquivos

API

```
Plain Text
```

```
lost-media-metasearcher/
├── src/
│   ├── config/
│   │   └── index.js           # Configurações globais
│   ├── controllers/
│   │   └── searchController.js # Orquestração da busca
│   ├── services/
│   │   ├── dorkEngine.js       # Geração de dorks
│   │   ├── scraperService.js   # Busca assíncrona
│   │   ├── linkResolver.js     # Resolução de links diretos
│   │   └── linkChecker.js      # Verificação de links
│   ├── utils/
│   │   └── errorHandler.js     # Tratamento de erros
│   └── app.js                  # Servidor Fastify
├── package.json
└── README.md
```

Frontend

Plain Text

```
widicom/
├── client/
│   ├── public/                 # Arquivos estáticos (favicon, etc.)
│   ├── src/
│   │   ├── components/
│   │   │   ├── SearchBar.tsx   # Barra de busca
│   │   │   └── ResultCard.tsx  # Card de resultado
│   │   ├── lib/
│   │   │   └── api.ts          # Serviço de API
│   │   ├── pages/
│   │   │   └── Home.tsx        # Página principal
│   │   ├── App.tsx             # Componente raiz
│   │   ├── main.tsx            # Ponto de entrada
│   │   └── index.css           # Estilos globais
│   └── index.html
├── package.json
└── .env.example
```

Próximos Passos e Melhorias Futuras

O Widicom é um projeto em desenvolvimento contínuo. Algumas melhorias planejadas incluem:

1. **Cache de Resultados**: Implementar cache para buscas frequentes, reduzindo o tempo de resposta.
2. **Histórico de Buscas**: Permitir que usuários visualizem e reutilizem buscas anteriores.
3. **Filtros Avançados**: Adicionar filtros por plataforma, data de atualização, tamanho de arquivo, etc.
4. **Suporte a Múltiplos Idiomas**: Expandir o suporte para buscas em diferentes idiomas.
5. **Integração com Notificações**: Alertar usuários quando novos resultados estão disponíveis para termos salvos.
6. **Dashboard de Estatísticas**: Mostrar estatísticas sobre as buscas mais populares e plataformas mais ativas.
7. **API de Webhooks**: Permitir que aplicações terceiras se inscrevam em eventos de busca.

Suporte e Contribuições

Se encontrar problemas ou tiver sugestões de melhorias, considere:

1. Verificar os logs da API e do frontend para mensagens de erro detalhadas.
2. Consultar a documentação do código-fonte (comentários em cada arquivo).
3. Abrir uma issue no repositório do projeto (se aplicável).

Licença

Este projeto é fornecido sob a licença MIT. Consulte o arquivo LICENSE para mais detalhes.

Desenvolvido com ❤️ por Manus AI

Última atualização: Março de 2026